AI Engineering Cohort - Assignment

Build a Workout Plan Generator

Objective

Build a single-page Streamlit app that takes structured inputs about a user's fitness goals and generates a personalized workout plan using an LLM via the Groq API. This assignment isn't about wiring up an API call, it's about designing inputs that give the LLM enough context to produce something a real person could actually follow.

The Use Case

A user should be able to open the app, tell it about themselves, and get back a usable weekly workout plan not a generic "do some squats" response. Think about what a personal trainer would actually ask before writing a plan for someone.

What to Build

1. Structured inputs (not just one text box)

At minimum, collect:

- Fitness goal (dropdown: Build muscle / Lose fat / General fitness / Improve endurance)
- Experience level (dropdown: Beginner / Intermediate / Advanced)
- Days available per week (slider or number input, e.g. 1–7)
- Equipment access (dropdown or multiselect: No equipment / Home dumbbells / Full gym)
- Optional: injuries or limitations (free text — e.g. "bad knees", "no overhead pressing")

2. A "Generate Plan" button

Sends these inputs to the LLM and displays the result in a clearly formatted area (e.g. a weekly breakdown, day by day).

3. A Python function with type hints

This function should:

- Take the structured inputs as parameters
- Build a well-designed prompt
- Call the Groq API and return the response
- Be wrapped in try/except

4. Basic error handling

- Missing/invalid inputs (e.g. 0 days selected) → friendly message, not a crash
- Failed API call (bad key, network issue, rate limit) → friendly message, not a crash

- Empty or malformed LLM response → friendly fallback message

---

The Actual Learning Goal: Prompt Design

Don't just concatenate the inputs into a sentence and hope for the best. Your prompt should push the model to:

- Respect the constraints (equipment, injuries, days/week) not ignore them
- Return a structured output (e.g. Day 1 / Day 2... with exercises, sets, reps) rather than a wall of text
- Stay appropriately scoped — no medical claims, and add a small disclaimer when injury-related input is given

Tip: iterate on your system prompt a few times with different inputs and see where the model ignores your constraints. That's the real exercise here.

Tech Stack

- Python - functions with type hints, try/except
- Streamlit for the UI
- Groq for the LLM call

Stretch Goals (optional, not required)

- “Regenerate” button to get a different variation of the plan
- Store the last generated plan in st.session_state so it persists across reruns
- Let the user download the plan as a .txt or .md file
- Add a “Swap this exercise” mini-feature for one exercise at a time

---

Rubric

Criteria	Weight
App runs without crashing on empty or invalid input	20%
Inputs are structured (not just free text) and correctly passed into the prompt	25%
Prompt design plan respects constraints, is well-structured, and is genuinely usable	30%
Error handling (API failure, empty/malformed response)	15%
Code quality (type hints, function separation, readability)	10%

Submission

In the session: **Session 1 - Kickoff and Fundamentals**, use the Submit button to submit the assignment.

Submit your GitHub repo link, make sure the repo is public so we can open and test it.